

Valknut Boss Battles Extension Report

Student: Hana Peročević

Extension: Boss Battles using the Template Method Pattern

Project: Valknut

Release: Hana_Recovery_Extension_v1.0

1. Introduction

This report describes the implementation of a Boss Battles extension for the Valknut game.

The purpose of the extension was to add boss enemies with special combat mechanics while preserving the original Valknut gameplay. The implementation uses the Template Method design pattern.

Two bosses were implemented:

- Fafnir
- Skoll

Both bosses follow the same general boss-turn structure, but each boss has its own special behavior.

Fafnir uses a two-phase combat system. In the first phase, he performs normal targeted attacks. In the second phase, he uses an area-damage fire attack before performing his normal attack.

Skoll uses a turn-based regeneration mechanic. He gathers moonlight during one turn and restores health during every second boss turn.

The extension is integrated into the existing story and combat flow. The player does not need to select a separate game mode or use an additional menu.

2. New User Story

2.1 User Story Information

User Story ID: US-BOSS-01

Title: Fight enemies with unique boss mechanics

Priority: High

Main actor: Player

2.2 Description

As a player, I want to fight boss enemies with special mechanics so that boss battles are different from normal combat and require more attention and strategy.

2.3 Preconditions

- The player has started a new game or loaded an existing game.
- The player has progressed through the story to a boss encounter.
- At least one hero is active.

2.4 Main Flow

1. The player reaches a boss battle through the normal story progression.
2. The player completes the available hero actions.
3. The boss starts its turn.
4. The boss announces its current phase or turn state.
5. The boss selects an active hero.
6. The boss applies its special mechanic.
7. The boss performs its normal attack.
8. The boss performs any effect required after the attack.
9. Combat continues until the heroes or enemies are defeated.

2.5 Acceptance Criteria

- Fafnir must appear automatically in the existing story.
- Skoll must appear automatically in the existing story.
- Fafnir must have at least two different combat phases.
- Fafnir must use an area-damage attack during Phase 2.
- Skoll must regenerate health during every second boss turn.
- Bosses must still use the existing targeting and attack functionality.
- Defend must continue to work against normal targeted boss attacks.
- Original enemies must continue to function normally.

- Boss battles must work when the game is started from the executable JAR.
- Boss objects must remain functional after saving and loading the game.

3. Design Pattern

3.1 Selected Pattern

The selected design pattern is the **Template Method Pattern**.

The Template Method Pattern defines the general structure of an algorithm in an abstract parent class. Individual steps can then be implemented or changed by subclasses.

In this extension, every boss follows the same general turn algorithm:

1. Announce the boss phase.
2. Select a target.
3. Apply a special mechanic.
4. Perform a normal attack.
5. Apply an after-turn effect.

This algorithm is implemented in the `performBossTurn()` method of the `AbstractBossEnemy` class.

Concrete boss classes can customize the individual steps, but they cannot change the overall order of the boss turn.

3.2 Why the Pattern Fits

Without the Template Method Pattern, each boss could contain a separate and duplicated implementation of the complete turn flow.

This would make the code more difficult to maintain because targeting, attacking, and special mechanics could be implemented differently in every boss class.

The Template Method Pattern provides one shared structure for all boss turns. New bosses can be added by extending `AbstractBossEnemy` and overriding only the necessary steps.

This makes the implementation easier to understand and extend.

3.3 Comparison with the RefactoringGuru Structure

The implementation follows the standard Template Method structure:

- `AbstractBossEnemy` represents the abstract class.
- `performBossTurn()` represents the template method.
- `announcePhase()` represents one customizable step.
- `applySpecialMechanic()` represents another customizable step.
- `performAttack()` provides shared attack behavior.
- `afterTurnEffect()` represents an optional hook or additional step.
- `FafnirBoss` and `SkollBoss` represent concrete subclasses.

The main difference is that the structure was adapted to the existing Valknut classes, especially `Enemy`, `Hero`, and `Combat`.

4. Implementation

4.1 AbstractBossEnemy

The `AbstractBossEnemy` class extends the original `Enemy` class.

Its main responsibility is to define the common boss-turn algorithm.

The method `performBossTurn()` controls the order of the boss actions. It calls the following steps:

- `announcePhase()`
- target selection
- `applySpecialMechanic()`
- `performAttack()`
- `afterTurnEffect()`

The class also checks whether there are active heroes before executing the boss turn.

The boss classes use the original hero and enemy functionality where possible. This reduces duplication and keeps the new extension connected to the existing project design.

4.2 FafnirBoss

`FafnirBoss` extends `AbstractBossEnemy`.

`Fafnir` has two phases.

Phase 1: Wing Assault

When Fafnir has more than 100 health points, the combat log displays:

FAFNIR - PHASE 1: WING ASSAULT

During this phase, Fafnir attacks one selected hero. He does not apply area damage to all heroes.

Phase 2: Inferno

When Fafnir has 100 health points or less, the combat log displays:

FAFNIR - PHASE 2: INFERNO

During this phase, Fafnir first applies fire damage to all active heroes. He then performs his normal targeted attack against one selected hero.

This creates a clear change in boss behavior based on the boss's current health.

4.3 SkollBoss

SkollBoss also extends AbstractBossEnemy.

Skoll keeps track of the number of boss turns that he has completed.

On odd-numbered boss turns, Skoll gathers moonlight but does not restore health.

On every second boss turn, Skoll activates Moonlight Regeneration and restores up to 20 health points.

The healing cannot increase Skoll's health above his maximum health.

After the regeneration mechanic, Skoll continues with his normal targeted attack.

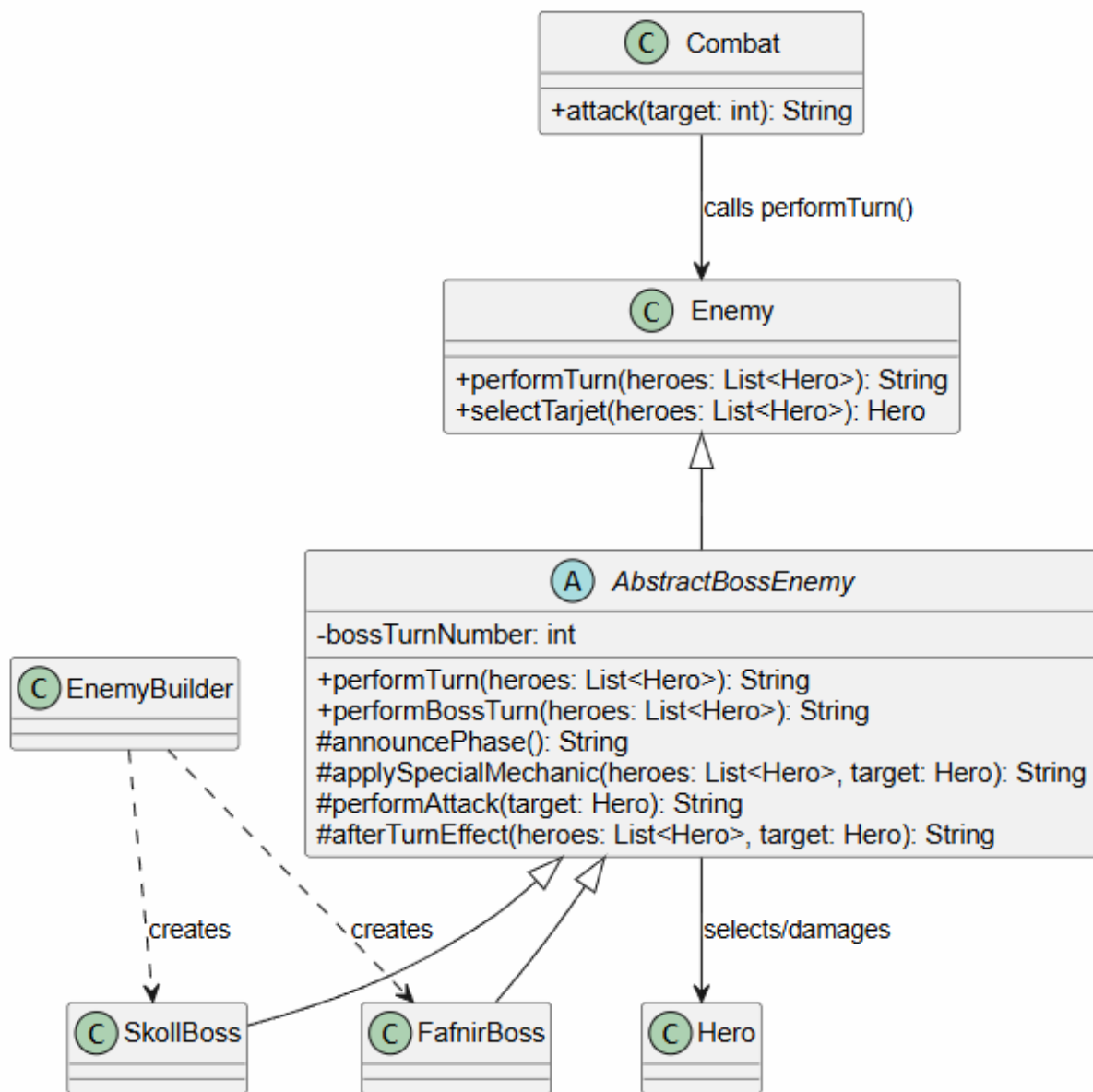
The boss-turn counter is part of the serialized boss object, allowing it to continue after saving and loading the game.

5. UML Diagrams

5.1 Class Diagram

The class diagram shows the relationship between the original Enemy class, the new AbstractBossEnemy class, and the concrete boss classes.

It also shows the connection with the Hero, Combat, and EnemyBuilder classes.

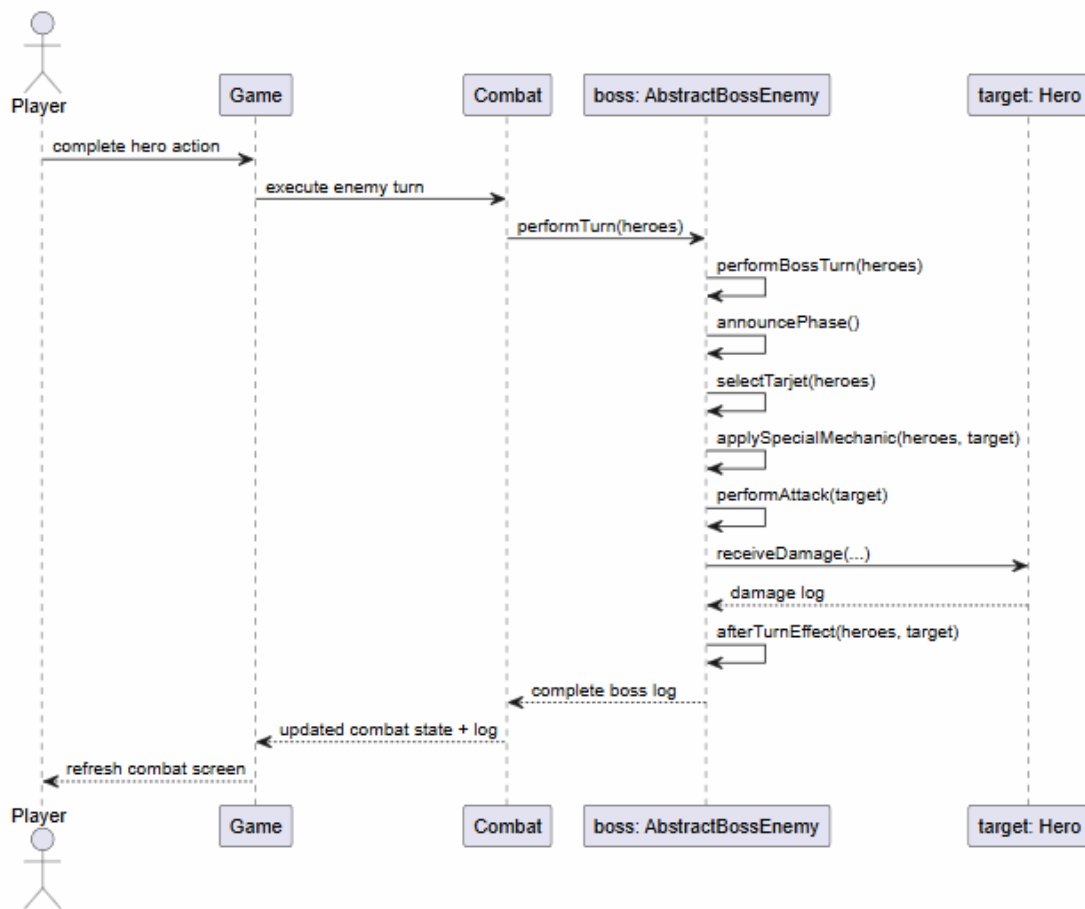


5.2 Sequence Diagram

The sequence diagram shows the main flow of a boss turn.

After the heroes complete their actions, **Combat** calls `performTurn()` on the current enemy.

For a boss object, this starts the Template Method sequence. The boss announces its phase, selects a hero, applies its special mechanic, performs its normal attack, and applies its after-turn effect.



6. Integration with Valknut

The extension was designed to preserve the original project structure and avoid unnecessary changes.

6.1 Enemy.java

A performTurn(List<Hero> heroes) method was added to the original Enemy class.

Normal enemies use this method to perform their existing targeting and attack behavior.

Boss classes override this method and use the boss-specific Template Method implementation.

This allows Combat to treat normal enemies and boss enemies through the same Enemy type.

6.2 Combat.java

The enemy-turn section of the combat flow was updated.

Instead of implementing enemy targeting and attacking directly inside Combat, the class now calls:

```
enemy.performTurn(heroes)
```

This uses polymorphism.

For a normal enemy, the standard enemy turn is executed. For a boss object, the boss-turn template is executed.

6.3 EnemyBuilder.java

The existing builder was updated so that:

- "fafnir" creates a FafnirBoss
- "skoll" creates a SkollBoss

The existing story can therefore continue requesting enemies by name without directly depending on the new boss classes.

6.4 Story Integration

The original story order was preserved.

Fafnir appears during the existing Fafnir encounter. Skoll appears during the existing Skoll and Hati encounter.

No additional boss-selection menu was added.

6.5 AudioManager.java

The audio error handling was updated to prevent the application from closing when a computer does not support a requested audio line or audio format.

This change does not alter the boss mechanics, but it improves the reliability of the executable JAR on different computers.

7. Testing Evidence

The extension was tested through the graphical user interface and through the executable JAR.

Ten manual tests were documented in MANUAL_TESTS_BOSS_EXTENSION.md.

The following functionality was tested:

Test	Result
JAR starts outside the IDE	Passed
Fafnir Phase 1	Passed
Fafnir Phase 2 transition	Passed
Fafnir area damage	Passed
Skoll first boss turn	Passed
Skoll regeneration	Passed
Defend against boss attacks	Passed
Original enemy behavior	Passed
Story integration	Passed
Save/load during a boss battle	Passed

The JAR was copied to a folder outside the IDE and started using:

```
java -jar Valknut_Hana_Recovery_Extension_v1.0.jar
```

The main menu opened successfully.

Fafnir was tested through the GUI. During Phase 2, the combat log displayed:

FAFNIR - PHASE 2: INFERNO

Both active heroes received fire damage before Fafnir performed his normal targeted attack.

Skoll was also tested through the GUI. His first turn did not restore health, while his second boss turn activated Moonlight Regeneration.

Optional: Insert the Fafnir Phase 2 screenshot here.

8. Known Limitations

The current extension has the following limitations:

- Only two concrete boss classes are implemented.
- Fafnir's phase transition uses a fixed health threshold.
- Skoll always regenerates on every second boss turn.
- Boss mechanics currently use combat-log messages instead of new animations.
- No separate boss health bar or phase indicator was added.
- No new boss sprites or sound effects were added.
- Game balance was tested manually but was not tested with a large number of automated simulations.
- Save/load functionality depends on the existing Java serialization system.

These limitations do not prevent the required functionality from working.

9. AI Use Declaration

ChatGPT was used to assist with the initial boss-class structure, suggestions for integrating the extension with the existing combat system, debugging support, documentation structure, UML planning, and manual test planning.

I reviewed and adapted the implementation to the existing Valknut project. I tested the functionality through the graphical user interface and through the executable JAR.

I verified the behavior of Fafnir's two phases, Fafnir's area-damage mechanic, Skoll's regeneration mechanic, the existing enemy behavior, story integration, and save/load functionality.

I understand the purpose and implementation of the following new classes:

- AbstractBossEnemy
- FafnirBoss
- SkollBoss

I also understand the modifications made to:

- Enemy
- Combat
- EnemyBuilder

- AudioManager

I can explain how the Template Method Pattern controls the order of a boss turn and how polymorphism allows the existing Combat class to work with both normal enemies and boss enemies.

10. Conclusion

The Boss Battles extension adds visible and functional boss mechanics to the original Valknut game.

The Template Method Pattern provides a shared boss-turn structure while allowing Fafnir and Skoll to implement different special mechanics.

The extension is integrated with the original combat and story flow. It does not require a separate game mode and does not remove the original enemy functionality.

Fafnir has a two-phase battle with an area-damage mechanic, while Skoll has a turn-based regeneration mechanic.

The extension works through the executable JAR and is supported by UML diagrams, documentation, and manual testing.